

COMP3016 report

The project I have created is a text-based bullet hell called DeffDred. The game has the player control a spaceship in a box shooting projectiles as enemies with a score increasing as time continues and as enemies die. The core objective of the game is to survive as long as possible and beat previous high scores. This game has an upgrade system to help diversify each game and make it feel different, each time you kill an enemy you will earn an amount of money, and when your money bar gets full you can choose between 1 of 3 upgrades presented to you. There are 3 enemy types and many upgrade types, with the upgrade types feeling rewarding and impactful, and the different enemies have unique moves and patterns that the player has to learn, as time continues, more enemies will spawn making your build really matter when it comes to the longevity of your run.

This project was made using modern C++17 and the standard library only, no external frameworks or graphics engines. It is fully self-contained and can compile on windows, linux or mac. The dependencies used were: `iostream`, `vector`, `memory`, `thread`, `chrono`, `algorithm`, `random`, `string` and `fstream`.

AI tools such as GitHub Copilot and ChatGPT were used as supporting assistants during development. Their role was to help research and understand specific functions, explore design ideas for OOP and game patterns and suggest improvements for cleaner and more refined code. All final code and logic were manually written and understood by me and AI was only used as a learning and explorational tool.

Some of the game programming patterns that I have used are entity component like hierarchy (inheritance) where I have player, enemy, rayenemy and boss all share a common interface for positioning, update logic and drawing. This creates polymorphism which means that the game can store all the active entities in one container and update them generically without each thing being treated individually. Enemies and bullets are dynamically spawned during runtime with varying attributes. Events are used like player stats like HP and money automatically trigger UI updates in the renderer. Singleton like control flow has the game glass handle initialization, main loops, updates and rendering as a single controller. Having smart pointer model makes it so that dynamic objects like bullets and upgrades use `std::unique_ptr` to prevent memory leaks and unusable references.

Player movement uses the classic WASD controls which use `GetAsyncKeyState()` which do not block inputs which allows the user to for example hold W and A and go diagonal instead of just going in the direction that was last pressed.

```
if (GetAsyncKeyState('W') & 0x8000) playerY--; (moving the player up in the Y axis for example by pressing W)
```

Each enemy updates independently every frame through a polymorphic `update()` method, standard enemies move lowly and randomly whereas ray enemies flash before firing and do not move while firing.

```
if (currentTime - lastFireTime >= fireInterval) {  
    bullets.push_back(Bullet(enemyX, enemyY, dx, dy));  
    lastFireTime = currentTime;  
}
```

collision is checked by comparing player and bullet positions but with an included small hitbox radius.

```
for (auto& bullet : bullets) {  
    if (abs(playerX - bullet.x) < 1 && abs(playerY - bullet.y) < 1) {  
        playerAlive = false; // player hit  
    }  
}
```

The screen rendering has the game board be a 2d array cleared and redrawn each frame. Once updated the screen is printed in one go using `std::cout` making smooth frame updates.

```
for (int y = 0; y < height; y++) {  
    for (int x = 0; x < width; x++) {  
        screen[y][x] = ' ';  
    }  
}  
  
screen[playerY][playerX] = '@';  
  
for (auto& b : bullets) screen[b.y][b.x] = '*';
```

during testing of the game, I had found out that having bullets need to hit the target exactly can be quite annoying and difficult since you have to be exactly precise and the smallest imperfection could cause you to miss. Because of this I had included a system where a bullet counts as hitting an enemy if the bullet hits any tile adjacent to the enemy itself, making the gameplay feel more forgiving and less irritating having an enemy dodge and weave the players attacks in an unfair fashion.

I have successfully created a fast-paced bullet hell game that has real-time input handling, dynamic object management, and clean ASCII rendering in C++. The project increased my understanding of OOP design, game loops, and data handling. If I were to improve it, I'd add more bullet patterns, scoring, and potentially colour effects for extra polish. I'm proud of achieving smooth gameplay and strong technical structure using only standard C++ features.

<https://github.com/AGoslin1/DeffDredCOMP3016>